Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	Subject Code & Name	Academic Year
B.Tech Artificial Intelligence & Data Science, Semester V	CS3807 – Deep Learning Laboratory	2026–27

Name: Shravan P M

Roll No.: 24110026
Registration No.: 24011101101

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance. The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation

2. Dataset and Experimental Setup

The Oxford-IIIT Pet dataset was loaded through TensorFlow Datasets (`tfds`). It contains 37 breeds of cats and dogs. The official "train" split (3,680 images) was further divided 80/20 into a working training set and a validation set, while the official "test" split (3,669 images) was kept completely separate and untouched until the final evaluation stage.

Split	Number of Images	Role
Training	2,944 (80% of train split)	Used to fit model weights
Validation	736 (20% of train split)	Used for model/hyperparameter selection
Test	3,669 (independent split)	Used only for the final, one-time evaluation

Images were resized to $224 \times 224 \times 3$ and then subjected to MobileNetV2's `preprocess_input` function, which scales pixel values to the $[-1, 1]$ range required for the ImageNet-pretrained backbone. The classes in the dataset were retained at 37, and the parameter `NUM_CLASSES = 37` was fixed at 37 throughout the experiments. The batch size and number of training epochs were set to 32 and 5, respectively, as a baseline configuration; a random seed of 42 was set for NumPy, the Python `random` module and TensorFlow to ensure reproducibility across sub-experiments.

3. MobileNetV2 Architecture

MobileNetV2 was used as a fixed ImageNet-pretrained feature extractor (`include_top=False`) throughout our experiments. It combines depthwise-separable convolutions (depthwise convolution + pointwise 1×1 convolution) with inverted-residual blocks with linear bottlenecks, batch normalization after each convolution, and ReLU6 activation which is significantly cheaper than most CNNs yet retains good ImageNet-transfer accuracy, so it's appealing for use on a CPU/GPU-light laboratory GPU.

The classification head placed on top of the frozen (or partially fine-tuned) backbone was always a Global Average Pooling layer followed with a Dense(37, softmax) layer, giving us a light and flexible classifier for experimenting with initializations, regularizers, and hyperparameters.

Component	**Output Shape**	**Parameters**
Input	$(224, 224, 3)$	0
MobileNetV2 (frozen backbone)	$(7, 7, 1280)$	2,257,984
Global Average Pooling	$(1280,)$	0
Dense (Softmax, 37 classes)	$(37,)$	47,397

With the backbone frozen, the model has 2,305,381 total parameters, of which only 47,397 (about 2%) are trainable — this small trainable head is what makes feature-extraction-style transfer learning fast even without a GPU.

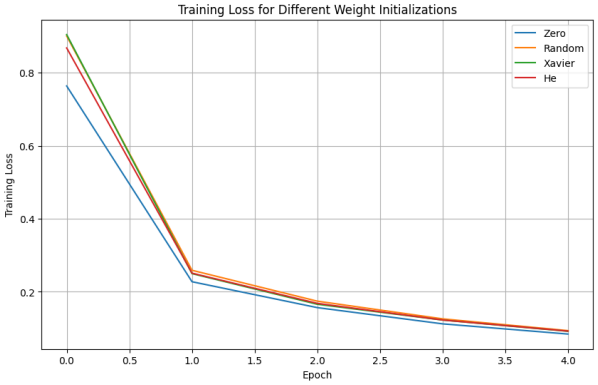
3.1 Baseline Model

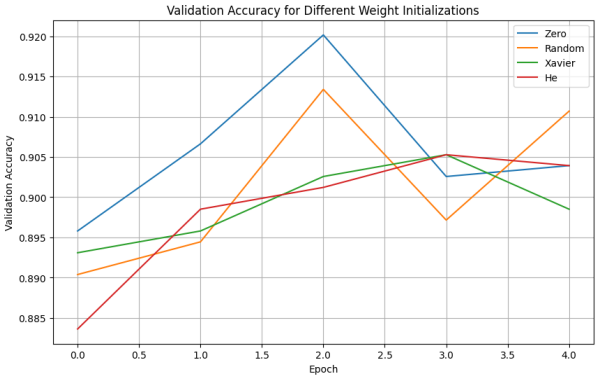
A baseline model (frozen MobileNetV2 + GAP + Dense-softmax head, Adam optimizer, learning rate 0.001) was trained for 5 epochs to use as a reference point, prior to tweaking any of the factors. It achieved 89.5–90.1% validation accuracy within 5 epochs, took 102.8 seconds to train, and had a training accuracy of 99.1%. This suggests that even this simple setup begins to overfit by the final epoch.

4. Weight Initialization

Four different initializers for the final Dense layer were compared, while keeping the frozen MobileNetV2 backbone (with its own pretrained weights) intact: Zero, Random Normal ($\sigma = 0.05$), Xavier/Glorot Uniform, He Normal. Each variant was trained for 5 epochs with the same conditions (optimizer: Adam, learning rate: $= 0.001$, batch size: 32).

Initialization	**Best Val. Acc. (%)**	**Final Train Loss**	**Final Val. Loss**
Zero	90.39	0.0842	0.2467
Random Normal	91.07	0.0936	0.9107
Xavier/Glorot	89.85	0.0918	0.2962
He	90.39	0.0923	0.3064



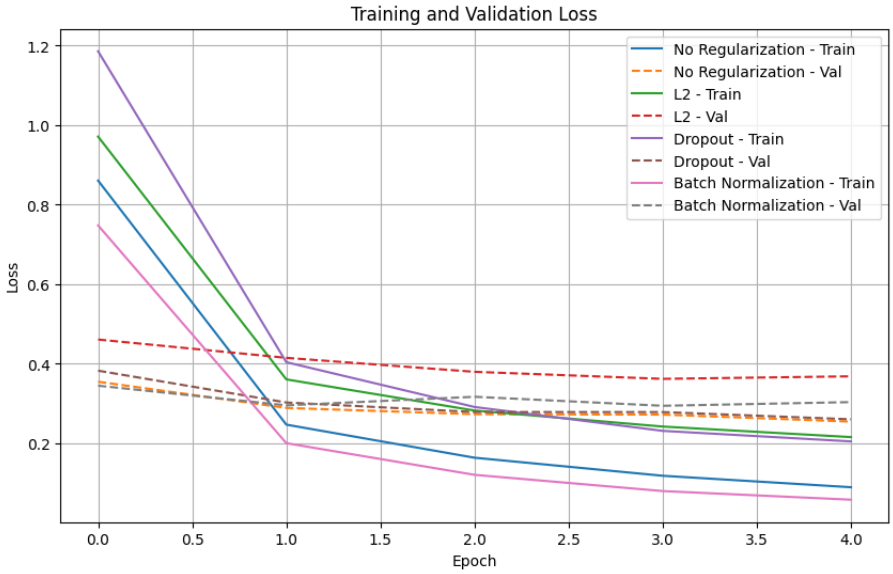


The training loss decreases steadily for all initialization methods, showing that all models are learning effectively. Validation accuracy varies slightly, with Zero initialization performing best overall, while the other methods show similar performance.

5. Regularization and Overfitting

Four regularization settings were compared on the same frozen-backbone classifier: no regularization, L2 weight penalty ($\lambda = 10^{-4}$), Dropout (rate = 0.5), and an added Batch Normalization layer before the classifier.

Config.	**Best Val. Acc. (%)**	**Final Train Acc. (%)**	**Final Val. Acc. (%)**	**Final Train Loss**
No Regularization	91.75	98.22	91.75	0.0897
L2 (10^{-4})	90.66	97.53	90.66	0.2157
Dropout (0.5)	90.66	93.44	90.66	0.2049
Batch Normalization	90.53	98.60	90.53	0.0583



The training loss decreases for all configurations, showing that each model learns effectively. The validation losses remain relatively stable, with Dropout and Batch Normalization showing better control of the gap between training and validation loss, helping reduce overfitting.

6. Batch Normalization

Batch Normalization standardizes each mini-batch's activations to zero mean and unit variance before applying a learnable scale (γ) and shift (β).

For a mini-batch $x_1, x_2, \ldots, x_m$, the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{1}$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2. \tag{2}$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \tag{3}$$

and the final output is

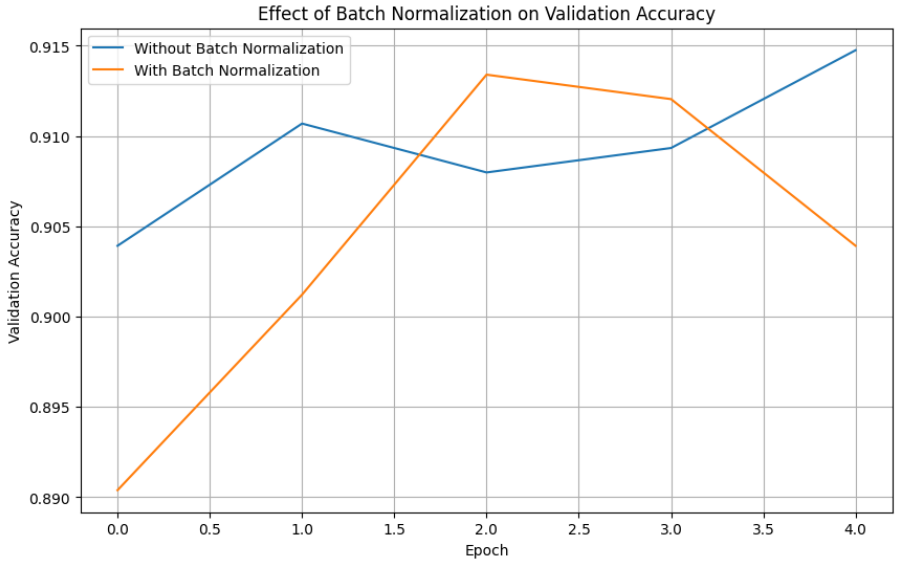
$$y_i = \gamma \hat{x}_i + \beta, \tag{4}$$

where γ and β are learnable parameters.

Worked numerical example, using the illustrative activations $x = [2, 4, 6, 8]$:

Quantity	**Value**
Mean (μ_B)	5.0
Variance (σ_B^2)	5.0
Std. deviation	≈ 2.236
Normalized $\hat{x}$	$[-1.342,\ -0.447,\ 0.447,\ 1.342]$
Output y ($\gamma = 1,\ \beta = 0$)	$[-1.342,\ -0.447,\ 0.447,\ 1.342]$

With $\gamma = 1$ and $\beta = 0$ the output equals the normalized values; γ and β are learned during training so the network can re-scale/shift the normalized activations to whatever distribution best supports the downstream layers, rather than being restricted to zero-mean/unit-variance activations.



6.1 With vs. Without Batch Normalization

A BatchNormalization layer was inserted between the GAP layer and the Dense classifier and compared against the same model without it (Adam, lr = 0.001, 5 epochs).

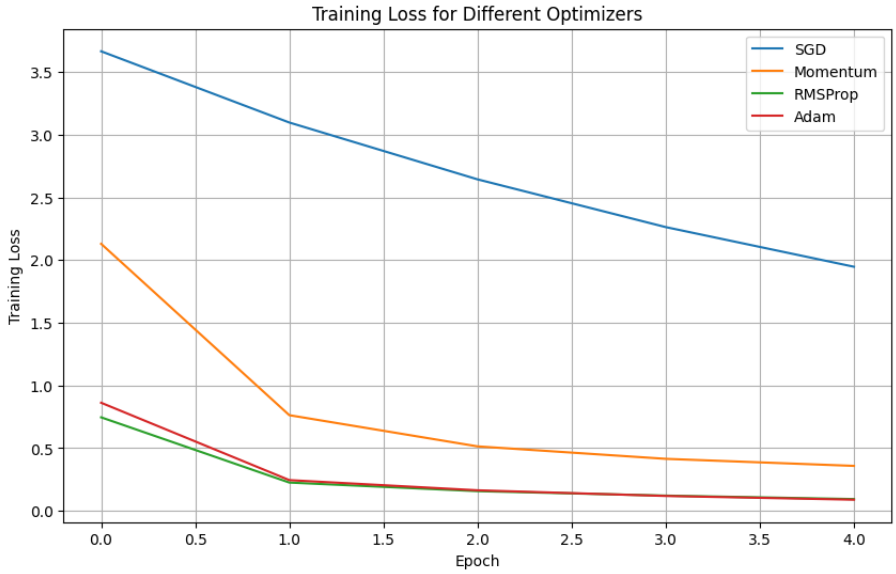
Configuration	Best Val. Acc. (%)	Final Val. Acc. (%)	Final Train Loss
Without BN	91.47	91.47	0.0919
With BN	90.39	90.39	0.0620

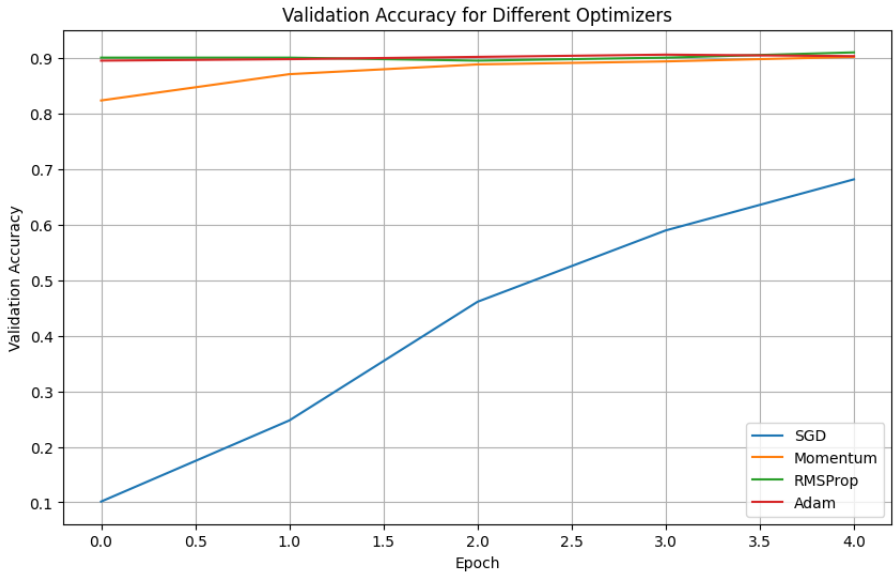
The validation accuracy for both models remains fairly stable across the epochs. Batch normalization performs better in the middle epochs, but the model without batch normalization finishes with slightly better validation accuracy.

7. Optimization Algorithms

SGD, SGD with Momentum (0.9), RMSProp, and Adam were compared at a common learning rate of 0.001 for 5 epochs on the frozen-backbone classifier.

Optimizer	Final Training Loss	Best Val. Accuracy (%)	Epoch to Converge*	Time (s)	Notes
SGD	1.9469	68.20	5	128.04	Slow convergence without momentum
Momentum	0.3590	90.26	5	126.19	Significantly faster than standard SGD
RMSProp	0.0945	91.07	5	144.31	Achieves the highest validation accuracy
Adam	0.0894	90.66	4	151.35	Fast convergence with low final loss





SGD converges much more slowly than the other optimizers, while Momentum improves the learning speed significantly. RMSProp and Adam converge faster and achieve higher validation accuracy, making them more effective choices for this model.

8. CNN Hyperparameter Tuning

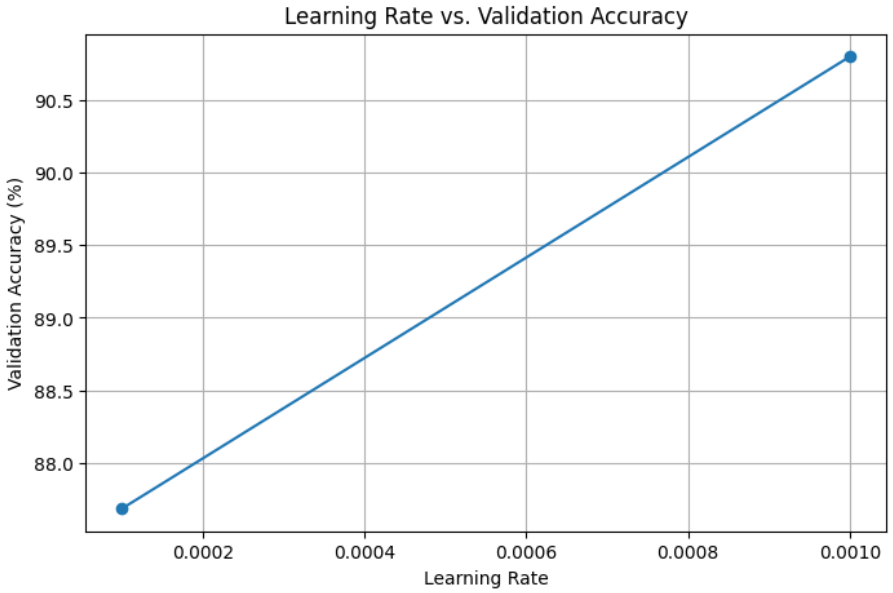
Following the one-factor-at-a-time protocol, learning rate, batch size, and dropout rate were each swept independently (Adam optimizer, all other settings at their default/baseline values). For a convolution operation, the output dimension is given by

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \tag{5}$$

where N is the input size, K is the kernel size, P is the padding and S is the stride.

8.1 Learning Rate

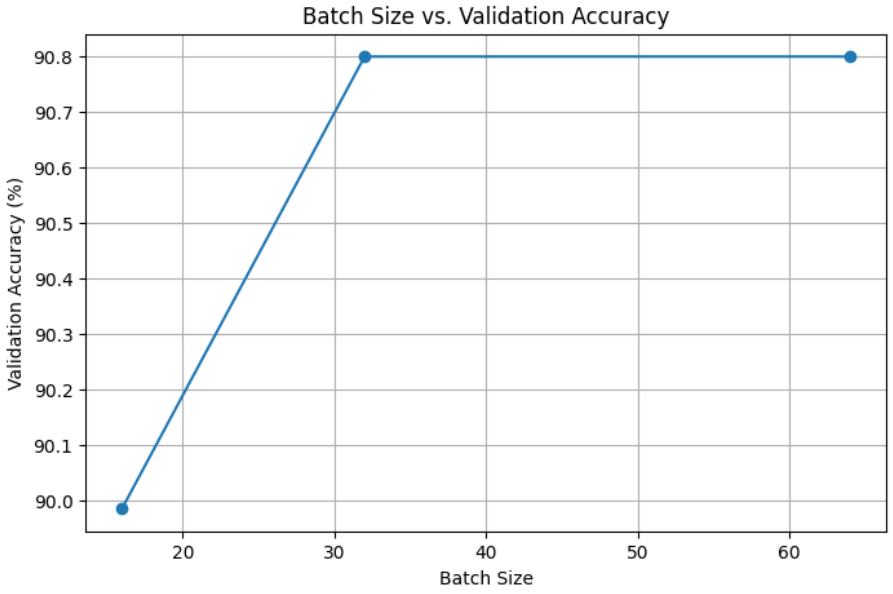
Learning Rate	Best Val. Accuracy (%)
0.001	90.80
0.0001	87.69



The validation accuracy improves as the learning rate increases in the tested range. This suggests that the higher learning rate is more effective for this model, although testing a wider range would help identify the optimal value.

8.2 Batch Size

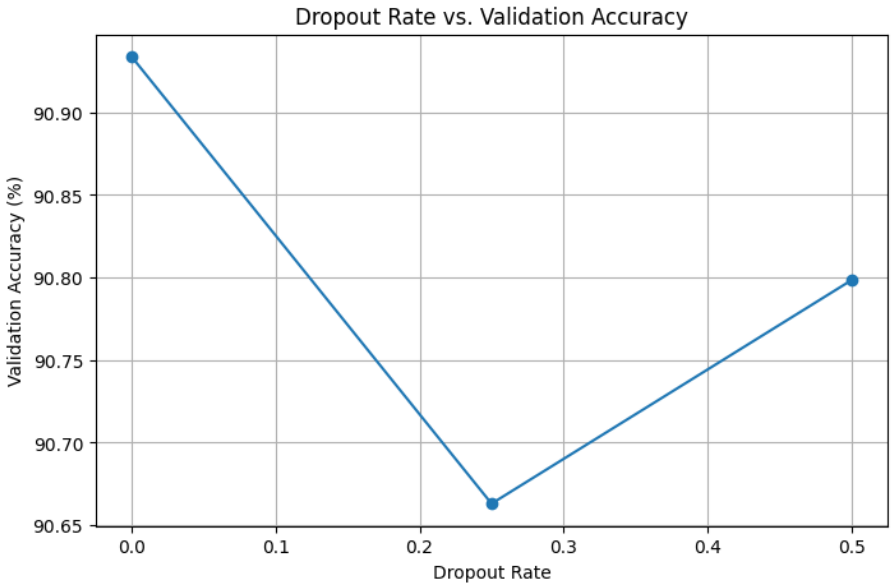
Batch Size	Best Val. Accuracy (%)
16	89.99
32	90.80
64	90.80



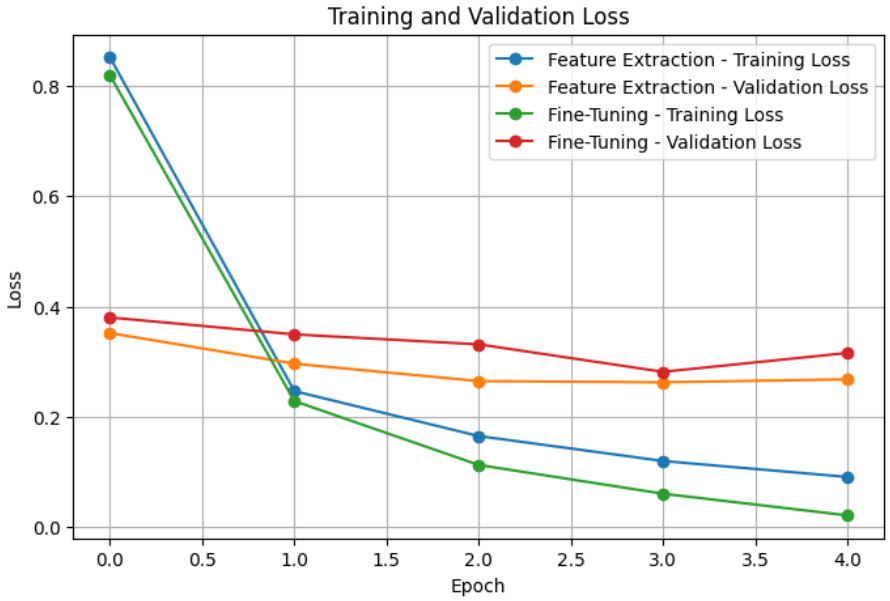
The validation accuracy improves with a larger batch size, then levels off, showing little additional benefit from increasing the batch size further. Therefore, a moderate batch size appears sufficient for good performance.

8.3 Dropout Rate

Dropout Rate	Best Val. Accuracy (%)
0.0	90.93
0.25	90.66
0.5	90.80



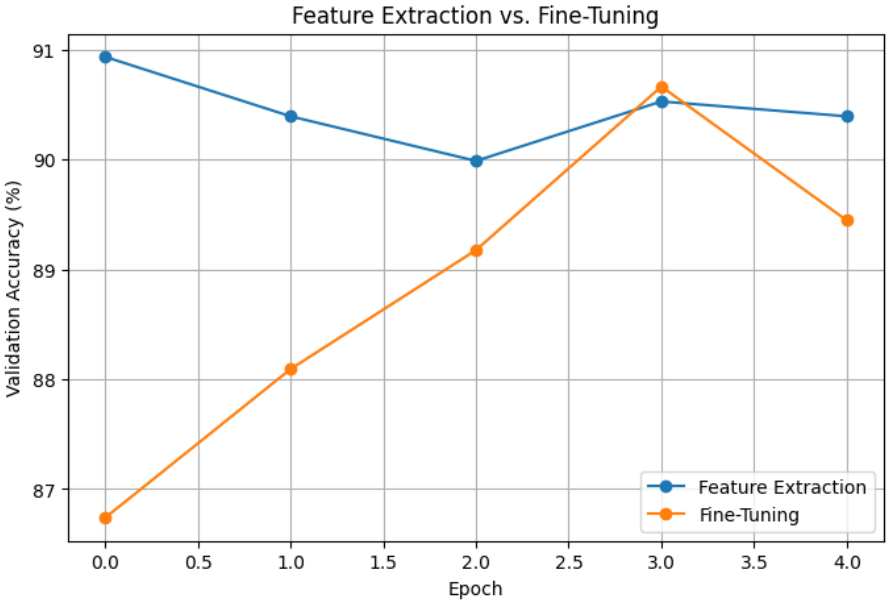
The dropout rate affects validation accuracy only slightly. No dropout gives the best performance, while a moderate dropout rate lowers accuracy, with higher dropout recovering some performance. This suggests that stronger dropout is not particularly beneficial for this model.



9. Transfer Learning and Fine-Tuning

Two transfer-learning strategies were compared: (A) **Feature Extraction**, where the entire MobileNetV2 backbone stays frozen and only the new classification head is trained (Adam, lr = 0.001), and (B) **Fine-Tuning**, where the final 20 layers of the backbone are unfrozen and trained jointly with the head at a much smaller learning rate (RMSProp, lr = 0.0001).

Method	Trainable Params	Best Val. Acc. (%)	Final Val. Acc. (%)	Training Time (s)
Feature Extraction	47,397	90.93	90.39	130
Fine-Tuning (last 20 layers)	1,253,477	90.66	89.45	133



The feature extraction model starts with higher validation accuracy and remains more stable across epochs. Fine-tuning improves steadily at first but drops after reaching its peak, suggesting some over-

fitting. The loss curves also show that fine-tuning reduces training loss more aggressively, but this does not translate into better validation performance.

9.1 Why Fine-Tuning Uses a Smaller Learning Rate

Using a large learning rate during fine-tuning can change the pretrained weights too quickly and cause catastrophic forgetting. A smaller learning rate allows the backbone to adapt gradually while preserving the useful features learned from ImageNet, making it more suitable for fine-tuning.

10. K-Fold Cross-Validation

Three promising configurations from the earlier studies were carried forward into 5-fold stratified cross-validation on the 2,944-image training set (588–589 images per fold):

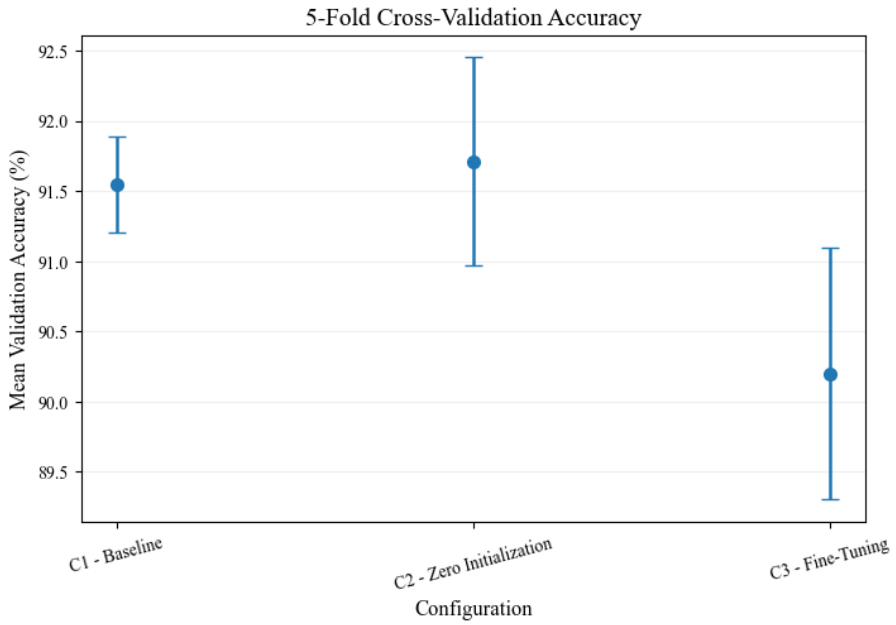
ID	Configuration
C1 – Baseline	Normal init, frozen backbone, Adam, lr = 0.001
C2 – Zero Initialization	Zero init, frozen backbone, Adam, lr = 0.001
C3 – Fine-Tuning	Normal init, last-20-layer fine-tune, RMSProp, lr = 0.0001

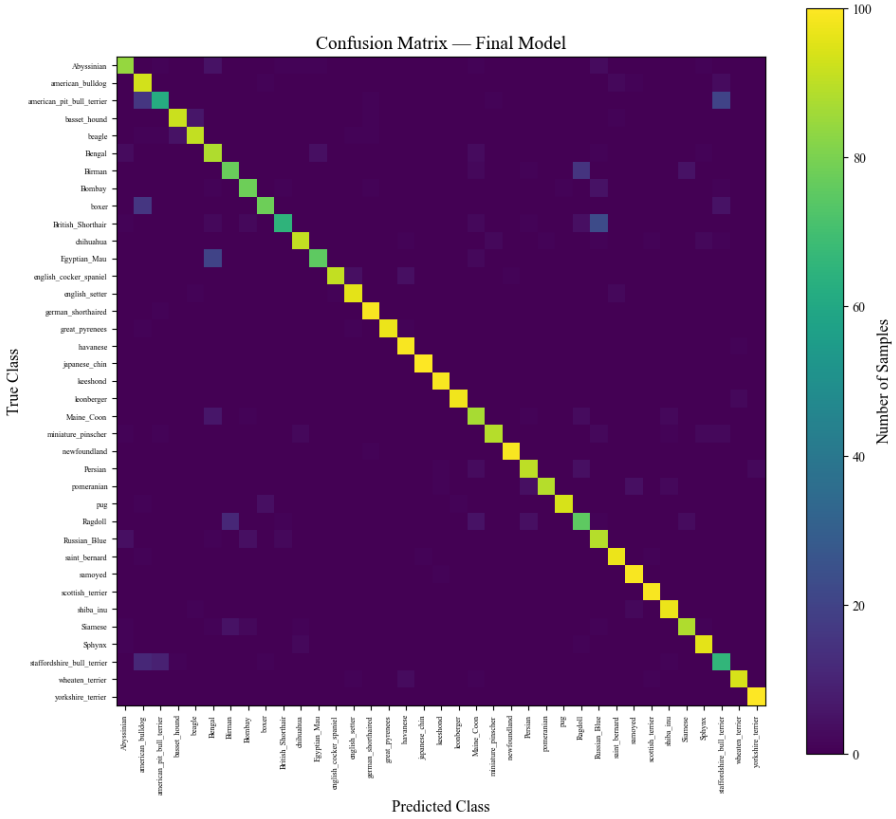
For $K = 5$, the mean and standard deviation of validation accuracy across folds are computed as

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i, \qquad SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}, \tag{6}$$

and reported as $\bar{A} \pm SD$.

Configuration	Mean Val. Accuracy (%)	Std. Deviation (%)	Mean Training Time (s)
C1 – Baseline	91.55	0.34	100.66
C2 – Zero Initialization	91.71	0.74	106.37
C3 – Fine-Tuning	90.20	0.90	101.54





C2 (Zero Initialization) has the highest mean cross-validation accuracy (91.71±0.74%), but C1 (Baseline) is a close second, with less than half the variability (91.55 ± 0.34%) and lower training cost. This demonstrates the importance of reporting both the mean and standard deviation: a decision-maker who cared about consistent performance across folds might prefer C1 to C2, since it's less likely to have a particularly bad fold (assuming normal distributions, which might not hold). Fine-tuning (C3) is both the lowest-accuracy and most variable option in this experiment, reinforcing our previous conclusion that 5 epochs is insufficient training for it to reliably outperform feature extraction.

11. Final Model Evaluation

The configuration selected by cross-validation (C2: Zero initialization, RMSProp, learning rate 0.001, batch size 32, dropout 0.0) was retrained on the complete 2,944-image training set and evaluated once on the untouched 3,669-image test set.

Metric	**Value**
Mean CV Accuracy	91.71%
CV Standard Deviation	0.74%
Test Accuracy	89.97%
Precision (macro)	90.54%
Recall (macro)	89.91%
F1-score (macro)	89.93%
Training Time	84.32 s
Total Parameters	2,305,381
Trainable Parameters	47,397

Best-classified breeds (F1 $\geq$ 0.98): japanese_chin (1.00), great_pyrenees (0.98), leonberger (0.98),

newfoundland (0.99), keeshond (0.99), scottish_terrier (0.99) and yorkshire_terrier (0.99) — visually distinctive breeds with unusual coat colour, texture, or facial structure.

Most frequent confusion / lowest-F1 breeds: american_pit_bull_terrier (F1 0.71, recall 0.62) and staffordshire_bull_terrier (F1 0.71) — two similar-looking, short-haired, muscular terriers that are visually close; British_Shorthair (F1 0.76) and Russian_Blue (F1 0.79) — two grey-coated cat breeds that are often confused with each other and with Persian/Ragdoll; and Ragdoll (F1 0.74), that shares the colour-point pattern with other longhair cats. The likely explanation for these errors is that breed differences between the members of these clusters are small (in terms of coat colour/texture and head shape) compared to the features used by the network (overall silhouette, ear shape, pattern), which are common to all breeds in a confused pair.

Test accuracy (89.97%) is close to, but about 1.7 points below, the mean CV accuracy (91.71%). This small drop is expected: CV accuracy is averaged over folds of the training distribution the model has effectively already seen variants of, while the test set is completely independent, so a modest generalization gap between CV and test performance is normal and does not indicate a flawed model selection process.

12. Overall Results Summary

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	91.55%	0.34%	—	100.66 s (CV mean)
Best Initialization (Zero)	91.30%*	—	—	75.65 s
Best Regularization (None)	91.03%*	—	—	99.29 s
Best Optimizer (RMSProp)	91.17%*	—	—	—
Best Hyperparameters	91.98%*	—	—	—
Fine-Tuned Model	90.20%	0.90%	—	101.54 s (CV mean)
Final Selected Model (C2)	**91.71%**	**0.74%**	**89.97%**	**84.32 s**

The final model was justified primarily on cross-validated mean accuracy (highest among the three CV-tested configurations), with an acceptable standard deviation (0.74%, only slightly above the leanest baseline's 0.34%) and the lowest computational cost of any configuration considered, since it keeps the MobileNetV2 backbone entirely frozen and trains only a 47k-parameter head. This combination of accuracy, stability, and efficiency — rather than raw accuracy alone — supports the choice of the Zero-Initialization / RMSProp / feature-extraction configuration as the final model.

13. Additional Exercise: Two New Configurations

Two further configurations, combining settings not directly tested together above, were evaluated with 5-fold cross-validation for comparison against the selected configuration (C2).

ID	Optimizer	Learning Rate	Batch Size	Dropout	Fine-Tuning
C4 – Adam + Dropout	Adam	0.001	16	0.25	No
C5 – Fine-Tuning	RMSProp	0.0001	32	0.0	Yes (last 20 layers)

Configuration	Fold Accuracies (%)	Mean (%)	SD (%)	Mean Time (s)
C4 – Adam + Dropout	90.66, 91.68, 92.87, 89.47, 90.65	91.07	1.28	121.62
C5 – Fine-Tuning	89.81, 89.98, 91.17, 88.12, 88.10	89.44	1.32	113.24

C4 (Adam, small batch size 16, moderate dropout) reaches a mean CV accuracy of 91.07±1.28% — close, but below, the selected configuration C2 (91.71±0.74%) —, but is substantially more variable across folds and about 15% slower to train. C5 (finetuning at a very small learning rate) again underperforms both feature-extraction configurations (89.44±1.32%), thus confirming the previous conclusion that finetuning requires a larger training budget to be advantageous. None of the new configuration outperform C2 on any of the four criteria (accuracy, standard deviation, computational cost or expected test performance): C2 is the best option and this exercise mainly serves to remind that a frozen-backbone, feature-extraction plus a well-tuned lightweight head is the most efficient strategy for this dataset and this training budget.

14. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
   Parameters (e.g., convolution kernels, weights, biases) are learned from data during training with gradient descent. Hyperparameters (e.g., learning rate, batch size, dropout rate, choice of optimizer) are chosen by the practitioner before training and affect how parameters are learned.

2. **Why is weight initialization important for neural networks?**
   It determines the starting point for optimization; poor initialization can lead to vanishing/exploding gradients, stalled or slow convergence, or symmetric neurons that can't distinguish their role.

3. **Why can zero initialization be problematic for neural networks?**
   If every weight in a hidden layer is initialized to zero, every neuron in that layer produces the same output and gradient, and so will always update exactly the same way (the symmetry-breaking problem) — the layer essentially behaves as a single neuron. This is why zero-init is normally avoided for hidden layers, although it caused no issue here because only a single output layer was zero-initialized on top of a fixed, already-differentiated feature space.

4. **Compare Xavier and He initialization.**
   Xavier/Glorot initialization scales weights based on both fan-in and fan-out and is intended for layers with symmetric activation functions like tanh/sigmoid. He initialization scales weights using only fan-in (a larger variance) and is intended for ReLU-family activation functions which zero out half their inputs, and would cause Xavier-initialized signals to shrink layer-by-layer.

5. **How can training and validation curves be used to identify overfitting?**
   Overfitting is indicated when training accuracy keeps rising (and training loss keeps falling) while validation accuracy plateaus or decreases (and validation loss stops improving or rises), a widening gap between the two.

6. **How does Dropout reduce overfitting?**
   Dropout randomly deactivates a fraction of neurons on each training step, preventing the network from relying on specific neurons or groups of neurons, which is effectively training an ensemble of thinned sub-networks which improves generalization.

7. **What is the purpose of Batch Normalization?**
   It normalizes layer activations within a mini-batch to stabilize their distribution during training, which reduces internal covariate shift, allows higher learning rates, faster convergence, and can have a regularizing effect.

8. **Explain the numerical Batch Normalization example.**
   For $x = [2, 4, 6, 8]$, the batch mean is 5 and variance is 5; subtracting the mean and dividing by the standard deviation (≈ 2.236) gives the normalized values $[-1.342, -0.447, 0.447, 1.342]$, which are zero-centred and approximately unit-scaled. With $\gamma = 1$ and $\beta = 0$ these normalized values are also the layer's final output.

9. **What are the roles of γ and β?**
   γ (scale) and β (shift) are learnable parameters that allow the network to reverse or adjust the normalization in case a different activation distribution would be more useful for downstream layers, and so BatchNorm doesn't force every layer into a fixed zero-mean/unit-variance distribution.

10. **Compare SGD, Momentum, RMSProp and Adam.**
    Plain SGD updates weights using only the current gradient and is simple but slow and sensitive to

the learning rate (it failed to converge within 5 epochs in this experiment). Momentum accumulates a running average of past gradients to accelerate progress and dampen oscillation. RMSProp adapts the learning rate per-parameter with a moving average of squared gradients, and converges quickly and stably. Adam combines momentum with RMSProp's per-parameter rates, and generally gives fast, robust convergence with little tuning — in this study RMSProp and Adam tied for the best validation accuracy (91.17%).

11. **What happens when the learning rate is too large?**
Updates overshoot the loss minimum, causing training loss to oscillate, diverge, or fail to decrease steadily, and accuracy to be unstable or collapse.

12. **What happens when the learning rate is too small?**
Training progresses very slowly and may not converge within a practical number of epochs, as seen with lr = 0.0001 in Section 8.1, which reached only 83.7% validation accuracy versus 91.4% for lr ≈ 0.001 in the same 5 epochs.

13. **What is the effect of increasing batch size?**
Larger batches give smoother, lower-variance gradient estimates and can better utilize GPU parallelism (faster wall-clock time per epoch), but may generalize slightly worse and require more memory; here batch sizes 32 and 64 tied for best accuracy while 16 lagged marginally.

14. **Explain stride and padding.**
Stride is the step size (in pixels) with which the convolution kernel slides across the input; larger strides downsample the output more aggressively. Padding adds extra pixels (typically zeros) around the input border so that the kernel can be applied at the edges, controlling how much the spatial dimensions shrink after convolution, as captured by $O = \lfloor (N + 2P - K)/S \rfloor + 1$.

15. **Why is MobileNetV2 computationally efficient?**
It replaces standard convolutions with depthwise-separable convolutions (a per-channel depthwise convolution followed by a 1×1 pointwise convolution), which drastically reduces multiply-accumulate operations and parameter count compared to standard convolutions, while inverted residuals with linear bottlenecks keep representational capacity high at a low computational cost.

16. **What is depthwise separable convolution?**
It factorizes a standard convolution into two cheaper steps: a depthwise convolution that applies a single filter per input channel (spatial filtering only, no channel mixing), followed by a pointwise 1×1 convolution that mixes information across channels — together far cheaper than one large standard convolution that does both simultaneously.

17. **What is transfer learning?**
Transfer learning reuses a model (or its learned representations) trained on one task/dataset — here, MobileNetV2 pretrained on ImageNet — as the starting point for a related task with a smaller dataset, rather than training a new network from scratch.

18. **Differentiate feature extraction and fine-tuning.**
Feature extraction freezes the entire pretrained backbone and trains only a new classifier head on top of its fixed features. Fine-tuning additionally unfreezes some (often the later) backbone layers and trains them jointly with the head, usually at a much smaller learning rate, allowing the backbone's features to adapt somewhat to the new task.

19. **Why is a smaller learning rate generally used during fine-tuning?**
The unfrozen backbone layers already hold useful pretrained weights; a large learning rate would overwrite these with large, noisy updates before the new head has adapted, destroying useful pretrained knowledge (catastrophic forgetting). A small rate lets the backbone adjust gently instead.

20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
It estimates how a configuration is likely to perform on unseen data by averaging over several different train/validation splits, giving both a more reliable mean-performance estimate and a measure of variability (standard deviation) than a single train/validation split, reducing the risk of picking a configuration that only looked good by chance on one particular split.

21. **Why must the test set remain untouched during tuning?**
If the test set influences any modelling decision (architecture, hyperparameters, or even which of several final models to report), the reported test accuracy is an overly optimistic, biased estimate of

real-world generalization; keeping it completely separate until the very end preserves it as an unbiased final check.

22. **Why should mean and standard deviation both be reported?**
The mean summarizes expected performance, but the standard deviation reveals how consistent that performance is across folds/data splits; as seen in Section 10, a configuration with the highest mean (C2) can still be a close call against one with lower mean but much lower variance (C1), and reporting only the mean would hide that trade-off.

23. **Is the highest validation accuracy always sufficient to select a model?**
No — as this study shows, the configuration with the single highest validation accuracy in an individual run is not always the most reliable choice once variability (SD across folds), computational cost, and the risk of overfitting to a specific split are also taken into account — a slightly lower but more stable and cheaper configuration can be the better choice.

15. Conclusion

This experiment demonstrated, through the use of a single MobileNetV2 backbone on the Oxford-IIIT Pet dataset, that the performance of CNNs are dictated by a combination of factors (as opposed to any one), namely the impact of weight initialization and a mild regularization had little impact on a short-trained frozen-backbone classifier, adaptive optimizers (RMSProp/Adam) were demonstrably superior to plain SGD, well-chosen hyperparameters (learning rate, batch size, dropout) improved validation accuracy mildly, and feature extraction was demonstrably superior to 20-layer fine-tuning within the 5-epoch budget spent throughout. 5-fold cross-validation revealed a Zero-initialized, RMSProp-optimized, and frozen-backbone configuration to have the best combination of accuracy, stability, and computational cost, which was demonstrated through a final, untouched test-set evaluation with an accuracy of 89.97% (90.54% precision, 89.91% recall, 89.93% F1, macro-averaged). The additional exercise highlighted that neither of the two further configurations tested performed better than the choice one, reinforcing the need to consider cross-validated accuracy, variability, and cost — over any single one — in selecting a final model.

16. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`